在 Context Logic 中表达 Noninterference

定义、Hypra 例子与可变变量

讨论总结

2026 年 10 月 7 日

摘要

本文只讨论 context logic 的断言与 capability 语义，不涉及 context type 或类型可靠性。首先给出 noninterference 的两种表达：利用原有的相容乘积 $*$ 定义取值一致性，以及引入带跨部分关系的连接词 $\oplus_R$。随后分析 Hypra 的 `secure` 和其 benchmark 中的 `DoubleSquare`，最后说明允许程序变量赋值时，如何用初始低输入快照与状态变换保持规格的正确含义。区分三类内容：原论文的逻辑定义、讨论中推导的派生公式，以及建议的语义扩展。

1 语义基础与目标性质

依照 Context Types 论文第 4 节 [1]，capability r 是一个非空环境集合，其中所有环境具有相同变量域。环境 σ 是从变量到值的部分映射。本文所有断言中涉及的程序变量均假设已绑定。

两个 capability 的相容性要求任意跨集合的两个环境相容：

$$r_1 \uparrow r_2 \iff \forall \sigma \in r_1, \tau \in r_2.\ \forall v \in \mathrm{dom}(r_1) \cap \mathrm{dom}(r_2).\ \sigma(v) = \tau(v).$$

相容时定义 $r_1 \times r_2 = \{\sigma \cup \tau \mid \sigma \in r_1, \tau \in r_2\}$。这里允许相容的重叠绑定，并非要求变量域严格不交。精化关系是投影关系：

$$r_1 \sqsubseteq r_2 \iff r_1 = r_2\big|_{\mathrm{dom}(r_1)}.$$

它保留已有变量的全部取值组合，仅允许增加新的变量绑定。

我们使用以下语义；D 简记论文中的 demonic modality：

$$\begin{aligned}
r \models P * Q &\iff \exists r_1, r_2.\ r_1 \uparrow r_2 \ \wedge\ r_1 \times r_2 \sqsubseteq r \ \wedge\ r_1 \models P \ \wedge\ r_2 \models Q,\\
r \models \mathrm{Atom}(\phi) &\iff r\big|_{\mathrm{FV}(\phi)} = \{\sigma \mid \mathrm{dom}(\sigma) = \mathrm{FV}(\phi),\ \sigma \models \phi\},\\
r \models \mathsf{D}P &\iff \exists r' \supseteq r.\ r' \models P,\\
r \models x \triangleright P &\iff \forall a \in \{\sigma(x) \mid \sigma \in r\}.\ \mathrm{Fib}(r, x = a) \models P[x \mapsto a].
\end{aligned}$$

其中 $\mathrm{Fib}(r, x = a) = \{\sigma \in r \mid \sigma(x) = a\}$。对于已绑定的普通状态公式 ϕ，$\mathsf{D}\,\mathrm{Atom}(\phi)$ 表达每个状态均满足 ϕ；单独的 $\mathrm{Atom}(\phi)$ 则有更强的精确覆盖含义，二者不能混用。

只考虑最终输出可观察时，noninterference 要求：初始低输入相同的两次执行，最终低输出相同。若程序保留低输入变量 l，则可以在结果 capability 上写成

$$\forall \sigma, \tau \in r_{\mathrm{out}}.\quad \sigma(l) = \tau(l) \implies \sigma(o) = \tau(o). \tag{1}$$

这是 ∀∀ hyperproperty。式中的状态量词属于元语言，不是把 context logic 的 $\forall x.P$ 解释为状态量词。

2 方式一：用原有相容乘积定义一致性

对于变量 v，定义派生公式

$$B_v \triangleq \mathsf{D}\,\mathrm{Atom}(v = v), \qquad \boxed{\mathsf{Low}(v) \triangleq B_v * B_v.}$$

$v = v$ 的支持域保留为 $\{v\}$；不能把它简化为无支持域的 $\top$。B_v 不限制具体取值，但要求 capability 绑定 v。

命题 1. *对于绑定了 v 的非空 capability，*

$$r \models \mathsf{Low}(v) \quad \Longleftrightarrow \quad \forall \sigma, \tau \in r.\ \sigma(v) = \tau(v).$$

证明. 若 $r \models B_v * B_v$，存在两个都绑定 v 的相容因子。相容性要求两因子中任意环境在 v 上相等；由于两因子均非空，所有环境在 v 上只能有一个共同值。乘积是 r 的精确投影，因此 r 的 v 也只能取该值。反过来，若共同值为 a，可取两个因子均为单例 capability $\{[v \mapsto a]\}$。它们相容，乘积就是 r 在 v 上的投影，且均满足 B_v。 □

因此定义

$$\boxed{\mathsf{NI}(l, o) \triangleq l \triangleright \mathsf{Low}(o).}$$

对于不同且已绑定的变量 l, o，展开 $\triangleright$ 与命题1：

$$\begin{aligned} r \models \mathsf{NI}(l, o) &\Longleftrightarrow \forall a \in r(l).\ \forall \sigma, \tau \in \mathrm{Fib}(r, l = a).\ \sigma(o) = \tau(o) \\ &\Longleftrightarrow \forall \sigma, \tau \in r.\ \sigma(l) = \tau(l) \Longrightarrow \sigma(o) = \tau(o). \end{aligned}$$

第一层全称来自对所有 l-fiber 的检查；第二层来自相容乘积强制的两两一致。

两种规格形式。 需要区分上述两种定义方案与下面两种规格写法：

$$\{\mathsf{Low}(l)\}\ C\ \{\mathsf{Low}(o)\} \qquad \text{以及} \qquad \mathrm{Exec}_C(r) \models l \triangleright \mathsf{Low}(o).$$

前者一次考虑低输入一致的输入集合；后者允许输入集合包含多个低输入值，在结果中分组检查。若 C 保留 l，且执行的集合语义是各输入状态结果的并集，则对每个输入 fiber 应用前者，可得到后者。花括号是元语言中的程序规格，不是论文已有的程序模态。

3 方式二：带跨部分关系的合并连接词

原论文的 $P \oplus Q$ 只要求 capability 可由分别满足 P, Q 的两个非空部分合并而成：

$$r \models P \oplus Q \Longleftrightarrow \exists r_1, r_2.\ r_1 + r_2 \sqsubseteq r \wedge r_1 \models P \wedge r_2 \models Q,$$

其中两部分同域，$+$ 为集合并，且不要求两部分互不相交。它没有显式约束两部分之间的状态关系。

作为可选扩展，为固定有限支持域的二状态关系 R 定义

$$r \models P \oplus_R Q \iff \exists r_1, r_2.\quad r_1 + r_2 \sqsubseteq r \wedge r_1 \models P \wedge r_2 \models Q \\ \wedge\, \forall \sigma \in r_1, \tau \in r_2.\, R(\sigma, \tau).$$

要求 R 引用的变量在两因子中都已绑定。令

$$E_v(\sigma,\tau) \triangleq \sigma(v) = \tau(v), \qquad \mathsf{Low}_R(v) \triangleq \top \oplus_{E_v} \top.$$

则同样有 $r \models \mathsf{Low}_R(v)$ 当且仅当 r 中所有状态的 v 相同。跨部分相等由定义直接给出；同一部分内的相等，可借另一非空部分的任一状态作为中介推出。反方向取 $r_1 = r_2 = r$ 即可。因此第二种表达是

$$\boxed{\mathsf{NI}_R(l,o) \triangleq l \triangleright (\top \oplus_{E_o} \top).}$$

适用边界。 这里的等价性使用了相等关系的传递性与两部分的非空性；不能据此声称 $\top \oplus_R \top$ 对任意 R 都等价于全体状态两两满足 R。$\oplus_R$ 是本讨论的扩展提案，尚非原论文或当前实现的原语。根据原有的重叠相容乘积定义，表达本文的 noninterference 并不必须增加它。若把乘积改成严格不交的资源分离，方式一的编码则不再成立。

4 Hypra Fig. 2: secure

Hypra[2] 的例子为：

```
secure(h, l):
    if h > 0:  o := 2*l
    else:      o := l
    if h <= 0: o := o+l
    return o
```

其规格对应 $\{\mathsf{Low}(l)\}\ C\ \{\mathsf{Low}(o)\}$。固定低输入为 a，两个阶段的值为

秘密分支	第一个条件语句后	最终输出
$h > 0$	$o = 2a$	$o = 2a$
$h \le 0$	$o = a$	$o = a + a = 2a$

中间不能要求 $\mathsf{Low}(o)$，但最终每个 $l = a$ 的 fiber 都满足 $\mathsf{Low}(o)$，故

$$\mathrm{Exec}_C(r) \models l \triangleright \mathsf{Low}(o).$$

若两个秘密分支都非空，可用扩展连接词显式记录中间的跨分支关系 $\sigma_+(l) = \sigma_-(l)$、$\sigma_+(o) = 2\sigma_-(o)$；第二个条件语句将它变成输出相等。若只有一个分支出现，应单独处理，不能用空 capability 充当 $\oplus$ 的因子。

现有 Rocq 证明的含义。 `secure_noninterference` 对任意绑定了 `high`、`low` 的输入 capability 证明结果满足 `FNoninterference low out`。证明先用 `FNoninterference_spec` 展开为两状态性质，再使用 `secure_result` 给出的 $\mathrm{secure}(h,l) = 2l$。这里程序是整数状态变换的语义模型，并非 CoreLang 中的程序项；关键的算术与分支分析来自单独的函数正确性证明。

5 非平凡 benchmark：DoubleSquare

Hypra artifact 的说明 [3] 确认其 benchmark 包含 `forall/pcsat/double_square_ni.hhl`。下列算法按上游 PCSat 文件中的程序整理 [4]；没有逐字核对虚拟机内的 Hypra 翻译文件，下述 context logic 不变式是本讨论的推导，而不是 Hypra 原注解。

```
DoubleSquare(h: bool, x: int):
    y := 0
    z := if h then 2*x else x
    while z > 0:
        z := z-1
        y := y+x
    if not h:
        y := 2*y
    return y
```

h 是秘密输入，x 是低输入，y 是可观察输出。当 $x = 3$，两种秘密分支分别运行 6 次和 3 次循环，循环结束时 y 分别为 18 和 9，最终都返回 18。目标仍是

$$\{\mathsf{Low}(x)\}\ C\ \{\mathsf{Low}(y)\}, \qquad \text{进而} \quad x \triangleright \mathsf{Low}(y).$$

5.1 表达式一致性与循环不变式

对整数表达式 e，用新鲜辅助变量 q 定义

$$\mathsf{Const}(e) \triangleq \exists q.\big(\mathsf{D}\,\mathrm{Atom}(q = e) \wedge \mathsf{Low}(q)\big).$$

这里使用原论文的存在量词语义，即增加一个变量绑定的 capability 扩展。$q = e$ 保留逐状态相关性，$\mathsf{Low}(q)$ 强制辅助值在所有状态间一致。因此

$$r \models \mathsf{Const}(e) \iff \forall \sigma, \tau \in r.\ [\![e]\!]_\sigma = [\![e]\!]_\tau.$$

辅助绑定的存在不等于存在一次程序执行，目标 hyperproperty 仍是全称性质。这是论文层面的派生定义；不声称当前 Rocq 核心已经提供该存在量词构造。

记 $z^+ = \max(z, 0)$，定义

$$E(h,x,y,z) = \begin{cases} y + xz^+, & h = \mathsf{true}, \\ 2(y + xz^+), & h = \mathsf{false}. \end{cases} \qquad \boxed{I \triangleq \mathsf{Low}(x) \wedge \mathsf{Const}(E).}$$

E 表示计入剩余工作与最后翻倍操作后的结果。条件表达式与 max 是状态表达式的简记，可以在 qualifier 中按条件展开，不是新增逻辑连接词。

5.2 初始化、保持与退出

初始化。 固定一个低输入值 $x = a$。初始化之后，两种分支均满足

$$E = 2a\max(a, 0).$$

真分支使用 $a\max(2a, 0) = 2a\max(a, 0)$；假分支直接得到同一值。所以存在共同辅助值 q，使 I 成立。

循环保持。 对活跃状态 $z > 0$，有 $z' = z - 1$、$y' = y + x$，以及

$$(z-1)^+ = z^+ - 1, \qquad y' + x(z')^+ = (y+x) + x(z^+ - 1) = y + xz^+.$$

h, x 不变，故 $E' = E$。所有状态仍可使用原来的共同辅助值 q。这证明的是保留同一个共同值，而不只是各个分支各自具有某个常量。

不同步退出。 在元语言中使用一步变换：对 $z > 0$ 的状态执行循环体，对 $z \le 0$ 的状态执行恒等变换。已完成的执行保留在集合中，不必与其他执行同步退出。每一步都保持 E，故保持整个集合的 I。对于固定 $x = a$ 的 fiber，所有执行至多需要 $2\max(a, 0)$ 步就有 $z \le 0$。对含多个、乃至无界多个低输入值的 capability，逐 fiber 应用此论证即可，不要求整个集合存在统一的有限迭代上界。

退出与输出。 退出时 $z^+ = 0$，因此 $E = y$ 或 $E = 2y$。最后一个条件语句恰好使 $y_{\text{final}} = E$，于是 $\mathsf{Const}(E)$ 推出 $\mathsf{Low}(y)$。最终得到 $x \triangleright \mathsf{Low}(y)$。当 $x \le 0$，输出为 0；当 $x > 0$，输出为 $2x^2$。所以不能在无输入范围约束时直接声称输出恒为 $2x^2$。

此证明关注最终值，不保证执行时间或执行轨迹不泄漏 h。它给出了 capability 语义上的循环论证，尚未构造一般的命令证明系统或该例的 Rocq 证明。

6 允许 Mutable Variable 后的改动

6.1 当前值不能替代初始低输入

原来的 $l \triangleright \mathsf{Low}(o)$ 对当前 l 分组。考虑

```
l := h;
o := h;
```

最终 $l = o$，所以结果满足 $l \triangleright \mathsf{Low}(o)$，但它显然泄漏初始秘密。例如两次初始 $l = 0$、初始 h 分别为 1 与 2 的执行，最终输出分别为 1 与 2。因此真正需要保留的是初始低输入的身份，而不是禁止修改所有输入。

6.2 不可修改的初始快照

用新鲜 ghost variable l_0 保存初始值：

$$\mathrm{Snap}(r) = \{\sigma[l_0 \mapsto \sigma(l)] \mid \sigma \in r\}.$$

程序可修改 h, l, o，但不能修改 l_0。结果规格改为

$$\boxed{l_0 \triangleright \mathsf{Low}(o).}$$

若最终 l 也公开，则规格是 $l_0 \triangleright (\mathsf{Low}(l) \wedge \mathsf{Low}(o))$。多个低输入可保存为一组快照，再使用迭代的 binding reference operator。仅为上述输出 noninterference 保存 l_0 就足够，不必保存 h_0。

快照必须保留 $l_0 = l$ 的逐状态相关性，不能把快照值集合与状态集合独立相乘。后者会把一次执行与另一次执行的初始输入错误配对。程序不修改 l_0 时，逐状态执行与按 l_0 取 fiber 相容，因而可以逐初始输入组验证。

6.3 赋值是状态变换，不是精化

对标量赋值定义

$$\mathrm{Assign}_{x,e}(r) = \{\sigma[x \mapsto [\![e]\!]_\sigma] \mid \sigma \in r\}.$$

右侧表达式在赋值前求值。已有变量的值被覆盖，其他变量与 ghost 快照保留。例如 $\{[x \mapsto 1], [x \mapsto 2]\}$ 经 $x := 0$ 变成 $\{[x \mapsto 0]\}$。这不是 $\sqsubseteq$ 扩展，因为旧变量的投影已经改变。因此原来的 Kripke monotonicity 不能用来证明赋值保持断言。

对于本文的总终止例子，可在元语言定义

$$\models \{P\}\ C\ \{Q\} \iff \forall r.\ r \models P \Longrightarrow \mathrm{Exec}_C(r) \models Q.$$

P, Q 仍是原有断言，新增的是命令语义与相应证明规则。推广到可能无终止结果的程序时，还需单独约定空结果集合的处理；不能直接把空集当作原论文中的非空 capability。

$\mathsf{Low}(x)$ 仍表示某个程序位置上当前 x 的一致性。$x := x + 1$ 保持它，$x := h$ 则可能破坏它。不可未经证明就对任意 context logic 公式套用文本替换的赋值规则：变量同时参与原子的支持域、$*$ 的重叠以及 $\triangleright$ 的分组。此外，现有 $*$ 是状态集合的相容乘积，不自动赋予可变位置的独占写权限。本文只处理标量变量，不引入指针、别名或堆权限语义。

6.4 修改低输入仍可满足 noninterference

考虑

```
ghost l0 := l;
l := l+1;
o := 2*l;
```

在初始快照 $l_0 = a$ 的 fiber 中，逐步有

$$l = a \quad \longrightarrow \quad l = a + 1 \quad \longrightarrow \quad o = 2(a + 1).$$

每个 fiber 的最终输出一致，故满足 $l_0 \triangleright \mathsf{Low}(o)$。这说明加入 mutable variable 后，必须新增的是历史快照与状态变换，不必为了表达本文的 noninterference 而更改 Low 或增加新的合并连接词。

7 结论与证明范围

原有相容乘积已经足以表达取值一致性，结合 fiber 操作即可表达 noninterference。$\oplus_R$ 提供一种显式记录跨部分关系的可选方案，但不是本文例子必需的表达能力扩展。`secure` 的核心事实是返回值只依赖低输入；`DoubleSquare` 的核心则是一个允许异步推进、跨执行共享的守恒表达式。mutable 情况下应始终按初始低输入快照分组。这些结论不自动给出类型推导算法、不自动发现不变式，也不构成 context type 的可靠性证明。

参考文献

[1] *Context Types: Bunched Modalities for Unifying Safety and Reachability.* 用户提供的匿名论文稿，2026 年 7 月版本。参见第 4.1–4.3 节，尤其 Definition 4.3。

[2] Thibault Dardinier, Anqi Li, Peter Müller. *Hypra: A Deductive Program Verifier for Hyper Hoare Logic*. PACMPL 8(OOPSLA2), 2024. Fig. 2, Sections 2 and 5. doi:10.1145/3689756.

[3] Dardinier, Li, Müller. *Hypra artifact*, 2024. README 中的 benchmark 说明与翻译说明。Zenodo record 12671562.

[4] PCSat / coar benchmark suite. `DoubleSquareNI_hFT.clp`，文件头给出未按秘密分支特化的原始程序。上游源文件.